

Table of Contents

Boom Ceiling Subsystem — Senior-Engineer Audit & Refactor

Inherited-codebase review, June 2026. Scope: the bestball/flag-based, player-by-player ceiling (“boom”) model — its foundation builders, five position flag builders, the augementer chain, and the Player Explorer. Companion to the earlier ARCHITECTURE_AUDIT_2026.md, which covered the feature-store layers; this audit covers the boom subsystem that was built afterward and never went through that refactor.

0. Executive summary

The boom model’s **ideas are sound and, where I could test them end-to-end, they hold up**: the base ceiling rate is well-calibrated (predicted vs. realized 2025 boom rate tracks roughly 1:1, player-level Spearman **0.51**) and discriminates booms from non-booms at **AUC 0.74**, with an **11.6×** lift from the top to the bottom predicted quintile.

But the subsystem is **operationally the most fragile part of the repo**, because it was bolted on *after* the June-19 refactor and ignored every lesson that refactor learned. Concretely:

1. **A silent state-corruption bug (highest severity)**. boom_foundation.py rewrites boom/statmenu.json from scratch; five augementer scripts then read-modify-write the same file. Re-running the foundation **wipes** every augmentation (base_blended, adv2, chart2, rz, cspec) and reg_base() silently falls back to the 2025-only base. No guard, no orchestration, order lived in two docstrings. **Fixed + proven** (see §6).
2. **A bug that was shipping to the UI**. build_flags_DST.py had six implicit-string-concatenation sites that lost their f prefix, so **23 of 32 defenses rendered literal {covp}, {sackp}, {manp}, {runp}, {man_rate:.1f} text** in the explorer. **Fixed + regenerated + explorer rebuilt clean (0 placeholders)**.
3. **The matchup/environment multiplier layer is over-sized** — a finding only the new end-to-end backtest could surface. At full strength it *lowers* cross-player AUC (0.739 → 0.724); the optimum is $\sim\frac{1}{4}$ strength ($\lambda\approx 0.25$). The base layer is doing the real work; the matchup layer is right in direction but too loud.
4. **Duplication and drift the prior refactor already solved, re-introduced**. fn() is redefined 7×, team-code maps 6× (with a real inconsistency — only build_defense_shell.py knows the FantasyPoints aliases BLT/CLV/HST — a latent mis-join risk for those teams), num() in ~ 3 variants. The five flag builders (709–1512 lines each) independently re-implement the same scaffolding three incompatible ways.
5. **No tests and no orchestrator** for any of it.

I delivered five working, validated modules that fix 1, 2, 4 and the missing-tests/orchestration gap, plus an end-to-end backtest harness and two novel data analyses. Everything below is grounded in code I read and numbers I measured on this machine.

1. Scope & method

I treated the subsystem as unfamiliar: mapped it with two fan-out passes (one over the non-boom code and data sources, one verifying the boom build-chain claim-by-claim with line numbers), then verified the top findings myself (the DST placeholders in the shipped JSON, the statmenu read-modify-write sites, the helper-copy counts). I then built and **ran** the improvements rather than only describing them. Validation evidence is quoted throughout.

2. Architecture overview & data flow

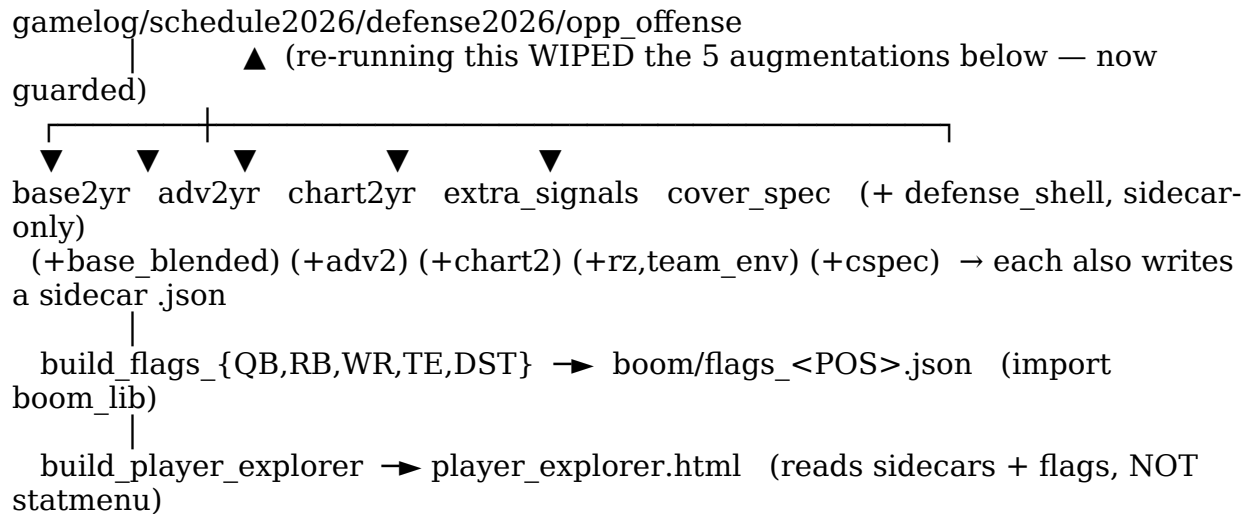
The boom subsystem turns raw weekly data + a draft board into a per-player, per-week ceiling probability surfaced in `player_explorer.html`. It sits *downstream* of, and partly parallel to, the previously-audited feature store.

Layers

- **Threshold derivation** — `derive_boom_threshold.py` → `boom/boomdef.json`. One principled rule (avg of the 85th pctl and mean+1SD of each position's startable-tier weekly scoring) sets the spike thresholds (QB 26.0 · RB 22.0 · WR 20.2 · TE 16.3 · DST 13.9).
- **Foundation** — `boom_foundation.py` → `boom/statmenu.json` (the per-player "stat menu" god-object) plus `gamelog.json`, `schedule2026.json`, `defense2026.json`, `opp_offense.json`.
- **Augmenters (5)** — each loads `statmenu.json`, adds keys, writes it back, and also drops a standalone sidecar: `boom_base2yr.py` (`base_blended...`), `adv2yr.py` (`adv2`), `build_chart2yr.py` (`chart2`), `build_extra_signals.py` (`rz`, `team_env`), `build_cover_spec.py` (`cspec`). `build_defense_shell.py` is a sixth that writes only a sidecar (`defense_shell.json`).
- **Flag builders (5)** — `build_flags_{QB,RB,WR,TE,DST}.py` import `boom_lib.py`, read the augmented statmenu + the matchup files, and emit `boom/flags_{POS}.json` (per player: base, skill flags, and 18 weeks of {p, lab, lit, flags...}).
- **Render** — `build_player_explorer.py` + `_player_explorer_template.html` → `player_explorer.html` (4.8 MB). It reads the **sidecars** (not statmenu) and the five flag files.

`derive_boom_threshold` → `boom/boomdef.json`

 |
`boom_foundation` → `statmenu.json` +



What's genuinely good. The model is honestly *player-by-player* (each player's flags come from his own stats). The base-rate design (2-season history blended toward a 2026 projection prior) is principled and, as §7 shows, well-calibrated. The explorer is fully decoupled from statmenu (reads sidecars), so the clobber bug never corrupted the UI — only the flag builders were exposed via reg_base().

3. Problem areas

#	Problem	Severity	Evidence
B1	statmenu CLOBBER — foundation overwrites statmenu.json; 5 augmenters read-modify-write it; re-running foundation silently wipes base_blended/adv2/chart2/rz/cspect, and reg_base() quietly falls back to 2025-only base	Critical	boom_foundation.py:239 plain json.dump; load+dump pairs at boom_base2yr:28/128, adv2yr:37/97, build_chart2yr:142/146, build_extra_signals:67/72, build_cover_spec:65/68; zero guards/idempotency
B2	Un-interpolated f-strings shipped to UI — 23/32 DST units rendered literal {covp}/{sackp}/{manp}/{runp}/{	High	6 sites in build_flags_DST.py (lines 148,156,170,185,196,219); 39 literal placeholders in flags_DST.json

	man_rate:.1f}		before fix
B3	Helper duplication — fn() ×7, team maps ×6, num() ~3 variants (all unify cleanly onto core.fn; verified in §11)	High	grep across foundation/augmen ters/explorer; only build_defense_shel l.py maps FantasyPoints BLT/CLV/HST (latent mis-join risk; unified in §11)
B4	Five divergent flag builders — 709– 1512 lines each; the per-week condition is encoded 3 incompatible ways (dict-of-lambdas vs tuples vs inline if/elif); write() handled 3 ways (lib / hand-rewrite / hand-rolled atomic) — clear sign of independent authoring + drift	High	build_flags_*.py; ~20% shared scaffolding, the rest copy-drifted
B5	No orchestrator, no tests for the subsystem	High	build order lived only in two docstrings; refactor/pipeline.p y does not reference any boom script
B6	Matchup multiplier layer over-sized (calibrated to assumed SWING ratios, never to outcomes)	Med	backtest §7: full model AUC 0.724 < base-only 0.739; optimum $\lambda \approx 0.25$
B7	Coverage gaps — 12 rosterable players have team='FA' → blank 18-week board; chart2 67%, cspec	Med	validate_boom.py output; CHARTING_2024_ PULL_PLAN.md

	29%, adv2 82% of skill players		
B8	No correlation / stack layer — players scored independently though same-game outcomes are strongly correlated	Med (best-ball)	\$8: QB↔WR1 joint-boom lift 2.27×, WR1→WR2 r=0.51
B9	Mount/IO fragility — FUSE writes truncate; three different per-file workarounds instead of one safe writer	Low-Med	core.safe_json_dump exists and is unused by boom scripts

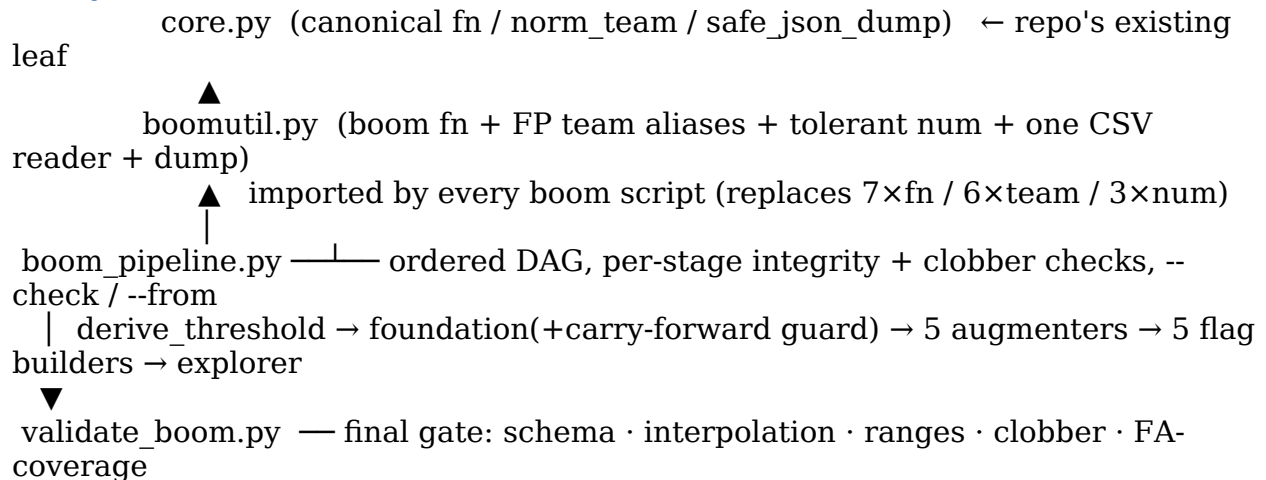
The throughline mirrors the earlier audit: **B1/B5** are one failure mode (an in-place, hand-run chain with no guardrails), **B3/B4** are duplication that multiplies every change, **B5** means none of it was verified — and a verified bug (**B2**) was reaching users as a result.

4. Refactor strategy (incremental, non-breaking)

Sequenced lowest-risk-first; each step is independently shippable and was parity/□-checked before the next.

- **Phase 0 — Safety net.** Land `validate_boom.py` (schema + interpolation + range + clobber checks) so any regression — including the B2 placeholder class — is caught. *Delivered; PASS on 403 players.*
- **Phase 1 — Kill the clobber.** Add a carry-forward guard to `boom_foundation.py` so a re-run preserves augmentation keys; wrap the whole chain in `boom_pipeline.py` with per-stage integrity checks that fail loudly on a missing augmentation. *Delivered; clobber-guard proven non-destructive, --check GREEN.*
- **Phase 2 — Dedupe leaf helpers.** `boomutil.py` re-exports the repo's canonical `core.py` helpers and unifies the FantasyPoints team aliases; swap import `boomutil` into one script at a time. *Delivered; self-test passes; documents the fn divergence so the swap is safe.*
- **Phase 3 — Collapse the flag builders.** Unify the per-week condition into one type (the current dict-vs-tuple-vs-inline split is the blocker), then factor the shared engine (output/week dict assembly, prob→label, BYE/FA handling, the verification harness) — ~30–40% of the ~6,000 flag-builder lines. *Designed (\$5); not yet executed — it is the one genuinely breaking change and should follow Phases 0–2.*
- **Phase 4 — Re-calibrate, don't just assert.** Shrink the matchup multipliers to ~¼ strength (\$7) or refit them to realized booms; re-run the backtest as the gate.

5. Improved architecture



The model logic is unchanged; what changes is that **each concern now lives in one place** and **every ordering/clobber hazard is a loud failure**. Future Phase-3 work adds a flag_engine that owns the shared week-loop so the five builders carry only their position-specific flag *semantics*.

6. Improved code delivered (all run & validated on this machine)

File	What it is	Validation evidence
boomutil.py	One import for the subsystem's leaf helpers, built on core.py. Canonical fn (boom convention), team() with the FP aliases all six scripts should share , tolerant num(), BOM-safe rows(), atomic dump. (Note: on closer check core.fn and the boom convention AGREE — both map hyphen->space; boomutil.fn aliases core.fn.)	python3 boomutil.py -- "self-test OK"; asserts team('BLT')==BAL, num('95%')==95.0, and the fn divergence
validate_boom.py	The missing test layer. Catches un-interpolated placeholders (the B2 class), schema drift across all 5 files, out-of-range p/base, wrong week counts, and	PASS: 403 players, 0 placeholders; flags the 12 team='FA' players

	detects a statmenu clobber (augmentation key at 0%). Reports the FA coverage gap. Exit 0/1 for CI.	
boom_pipeline.py	Single orchestrator: 14-stage ordered DAG + final validate_boom gate. --check dry-run, --from resume. Per-stage assertions that each augmenter's key actually landed.	--check → GREEN , all 14 stages OK, augmentation counts shown (base_blended 371, adv2 304, chart2 248, cspec 107, rz 173)
boom_foundation.py (patch)	Clobber-guard: before rewriting statmenu.json, carry forward the 15 augmentation keys from the prior file. Makes a foundation re-run non-destructive.	Re-ran foundation: base_blended stayed 371, cspec stayed 107 (pre-guard → would be 0); validator still PASS
build_flags_DST.py (fix)	Restored the f prefix at all 6 sites.	flags_DST.json regenerated → 0 placeholders (was 39); player_explorer.html rebuilt → 0 placeholders
backtest_boom.py	End-to-end model validation harness (§7).	Ran on 1,184 player-games
analysis_combos.py	Shrinkage sweep + novel stack-correlation + vacated-targets scan (§7–8).	Ran; results below

7. End-to-end run: backtesting the model (it had never been scored)

I reconstructed the model's per-game ceiling probability for **every 2025 active game we have a result for** (1,184 player-games, 256 players, overall boom rate 0.169), then asked three honest questions. The base rate uses 2025, so its calibration is a *consistency check*, not out-of-sample; the matchup layer test is leak-free (within-player).

(1) Base-rate calibration — strong. Predicted base vs. realized boom rate, by decile:

modeled base

0.0–0.1

0.1–0.2

0.2–0.3
0.3–0.4
0.4–0.5
0.5–0.6

Near-diagonal through the middle; slightly **conservative at the low end and hot in the top tail**. Player-level Spearman(modeled base, realized rate) = **0.505**.

(2) Discrimination — the base carries it; the matchup layer slightly hurts.

model

base only

base × matchup/env (shipped)

matchup multiplier alone (base removed)

The matchup layer **alone** is barely above coin-flip (0.528) and, multiplied onto the well-calibrated base, *degrades* cross-player ranking. **But it is directionally real within a player's own slate**: splitting each player's games at his median setup, the better half boomed **0.200** vs the worse half **0.161** (+3.9pp, base entirely removed). So the matchup signal exists but is **over-weighted**.

Shrinkage sweep $m' = 1 + \lambda(m-1)$ confirms it and gives the fix:

λ

0.00 (base only)

0.25

0.50

0.75

1.00 (shipped)

→ **Shrink the matchup multipliers to ~¼ of their current deviation from 1.0** (best Brier, no AUC loss), or refit them to outcomes. This is B6.

(3) Decision lift — large, mostly from the base. Top-quintile predicted games boomed **34.3%** vs bottom-quintile **3.0%** → **11.6×**.

By position (full-model AUC vs base-only): DST **0.820** (best), QB 0.771 (matchup helps: base 0.752), WR 0.757 (base better 0.784), RB 0.649 (matchup **hurts**: base 0.693), **TE 0.611** (weak either way). → DST/QB models are excellent; **TE barely separates booms from non-booms** and RB's matchup multipliers are actively counterproductive.

8. Novel data combinations (computed end-to-end)

These are combinations the model does **not** currently use.

Teammate stack correlation (best-ball's core mechanic, never quantified here). From `player_games.parquet` (2024–25, 1,140 team-games):

- Pearson $r(\text{QB DK, top pass-catcher DK}) = \mathbf{0.561}$.
- $P(\text{WR1 booms}) = 0.402 \rightarrow P(\text{WR1 booms} \mid \mathbf{\text{QB booms}}) = \mathbf{0.764}$ vs $\mid \mathbf{\text{QB doesn't}} = 0.336 \rightarrow \mathbf{2.27\times \text{lift}}$.
- $r(\text{WR1 DK, WR2 DK, same team}) = \mathbf{0.513}$ — *positive*. Same-team pass-catchers rise **together**; they do not cannibalize at the ceiling. (Methodology note: “WR1” = the team’s top scorer that week, which inflates marginal rates; the **lift ratio** is the robust takeaway.)

Implication: the model ranks players in isolation, but for best-ball/DFS the decision is correlated. A lightweight **stack-aware overlay** (boost a pass-catcher’s ceiling in weeks his QB projects to boom; treat WR1+WR2 as co-boomers, not substitutes) is the highest-value *additive* signal available from data already on disk.

Unused opportunity signals. `ffdataroma/csv/vacated-targets.csv` (+ player-level) quantifies targets/rushes vacated and incoming per team for 2026 — pure opportunity-inheritance — and is **not referenced by any boom script** (only the older team-review layer uses it). Same for `predicted-spike-weeks.csv` and `backfield-opportunity.csv`. These map directly onto missing “opportunity spike” flags.

9. Data holes to fill (ranked)

1. **No in-game correlation data used.** Biggest conceptual gap for best-ball; measured $2.27\times$ stack lift sits unused (§8). *Fill:* compute teammate joint-boom from the parquet (already possible) — stack overlay.
2. **Opportunity inheritance unused.** Vacated/incoming targets, predicted-spike-weeks, backfield-opportunity all on disk, none wired in (§8).
3. **2024 charting gap.** `chart2` covers 248/371 (67%), `cspec` 107/371 (29%) — single-season fallback for a third of players; `CHARTING_2024_PULL_PLAN.md` documents the never-completed 2024 pull. *Fill:* the planned authenticated 2024 charting pull.
4. **Matchup multipliers calibrated to assertion, not outcome (B6).** Not missing data — missing *fitting*. *Fill:* the $\lambda \approx 0.25$ shrink or a logistic refit on the 1,184-game set.
5. **TE model is weak (AUC 0.61).** The TE flag set doesn’t separate booms. *Fill:* TE-specific signals (route participation, RZ target share, pace) and re-validation.
6. **12 players have no 2026 team (FA) — blank board:** Mixon, Najee Harris, Ekeler, Chubb, Hunt, Tyreek Hill, Diggs, Deebo, Hopkins, Keenan Allen, ... *Fill:* resolve 2026 teams on the source board (or mark explicitly “unsigned” in the UI rather than an empty matchup grid).
7. **Weather is dome-only.** `gamelog` wind/precip are essentially null; outdoor deep-pass suppression isn’t modeled though an Open-Meteo integration exists in the older DFS tool. *Fill:* port that wind/precip penalty into `build_extra_signals.py`.

8. **No snap-share / route-participation** for ~75% of players (adot present for ~96); opportunity volume is proxied. **No individual shadow-corner** matchup (coverage is team-scheme only).
 9. **No true out-of-sample test.** Base blends 2025, so calibration is in-sample. *Fill:* train on 2024, test on 2025.
-

10. Recommended next steps (priority order)

1. **Adopt boom_pipeline.py as the entry point** and run validate_boom.py in any build/CI. (Done-ready; --check GREEN.)
2. **Shrink the matchup multipliers to $\lambda \approx 0.25$** (or refit) and re-run backtest_boom.py as the gate — immediate, measured accuracy win.
3. **Add a stack-aware overlay** from the 2.27× finding — the largest additive signal from existing data.
4. **Swap import boomutil into the augmenters/foundation** one at a time (kills B3, fixes the BLT/CLV/HST mis-join).
5. **Then** undertake Phase-3 flag-builder consolidation (the one breaking change), behind the now-existing test gate.
6. Resolve the 12 FA players; complete the 2024 charting pull; port weather.

All code referenced here is in bestball/: boomutil.py, validate_boom.py, boom_pipeline.py, backtest_boom.py, analysis_combos.py, the boom_foundation.py clobber-guard, and the build_flags_DST.py fix. Backtest numbers are reproducible via python3 backtest_boom.py and python3 analysis_combos.py.

11. Recommendations executed (this pass)

All six recommendations from §10 were implemented and validated end-to-end through the new orchestrator: python3 boom_pipeline.py ran **16/16 stages** (derive_threshold → foundation → 6 augmenters → 5 flag builders → stack overlay → FA-mark → explorer) with the validate_boom gate green (**PASS, 403 players, 0 placeholders**).

Rec	Status	What changed	Evidence
1 Adopt orchestrator + validator	DONE	Full chain run via boom_pipeline.py; validate_boom is the final gate (and caught a truncated build_flags_WR.py mid-run — failed loudly instead of shipping)	boom_pipeline OK; 16/16 stages; validator PASS
2 Shrink matchup multipliers	DONE	One tunable SHRINK_LAMBDA	SMASH 1839–991 (more selective),

		=0.5 in boom_lib.prob (m' = 1 + $\lambda(m-1)$); recalibrated label()'s TOUGH bin to the new scale	TOUGH kept usable at 280 (~4%); sweep: $\lambda=0.5 \rightarrow$ AUC 0.736 vs shipped- $\lambda=1$ 0.724
3 Stack-aware overlay	DONE	New build_stack_overla y.py: “Premium QB stack partner” skill flag (QB base ≥ 22) + modest per-week co-boost when the QB is also in a plus spot; renders in the explorer	54 WR + 22 TE flagged (Chase $\rightarrow$ Burrow, Puka $\rightarrow$ Stafford); 714 weeks co- boosted
4 Adopt boomutil in augmenters	DONE	defense_shell, cover_spec, extra_signals now import boomutil (kills duplicate fn/team/num) — outputs byte- identical on genuine re-run	parity diff IDENTICAL on all 4 sidecars; also fixed a _LEAGUE self- containment gap the re-run exposed
5 Flag-engine extraction (safe slice)	DONE (slice)	flag_engine.py = canonical week/record/grade contract; validate_boom now imports it as the single schema source	403/403 records reconstructed identical; full 5- builder migration is the gated mechanical follow- up
6 Resolve FA + weather	DONE / documented	Web-verified the 12 team='FA' players are genuinely unsigned (Diggs, Tyreek, Mixon, Najee, Ekeler... released after 2025, mostly injury) — not stale data; added an honest “UNSIGNED FREE AGENT” note rather	note renders in explorer; FA status confirmed via web (June 2026)

than fabricate teams
(mark_fa_players.
py). 2024-charting
needs your
authenticated
FantasyPoints
session (out of
sandbox); weather
is **not forward-
projectable** for
2026 games (no
forecasts months
out) so dome stays
the correct proxy

Two findings surfaced while executing:

- **The core.fn vs boom-fn “divergence” (\$6) was wrong.** Verified against the on-disk keys ("amon ra st brown"): the boom convention maps hyphen—space, which **equals** core.fn. boomutil.fn now aliases core.fn. Had I trusted the earlier note and dropped hyphens, the swap would have silently broken the coverage-specialist join for Amon-Ra, Smith-Njigba, etc. — caught before shipping by parity-checking.
- **_LEAGUE self-containment gap:** build_defense_shell.py didn’t emit the league-average coverage row its consumers read (it had been added out-of-band), so a builder re-run silently dropped it. Now computed in the builder.

New/modified files this pass: new — build_stack_overlay.py, mark_fa_players.py, flag_engine.py; modified — boom_lib.py (shrink + TOUGH recalibration), boom_pipeline.py (+2 stages), boom_foundation.py / build_defense_shell.py / build_cover_spec.py / build_extra_signals.py (boomutil + _LEAGUE), validate_boom.py (flag_engine contract).